Distributed file systems

Distributed Systems

Alberto Lafuente, Mikel Larrea
UPV/EHU

Contents

- Introduction
- System model
- Name service
- File service
- Examples:
  - Network File System (NFS)
  - Andrew File System (AFS)
  - Coda (COnstant Data Availability)
  - Common Internet File System (CIFS)
    - Server Message Block (SMB), *Samba*

Introduction

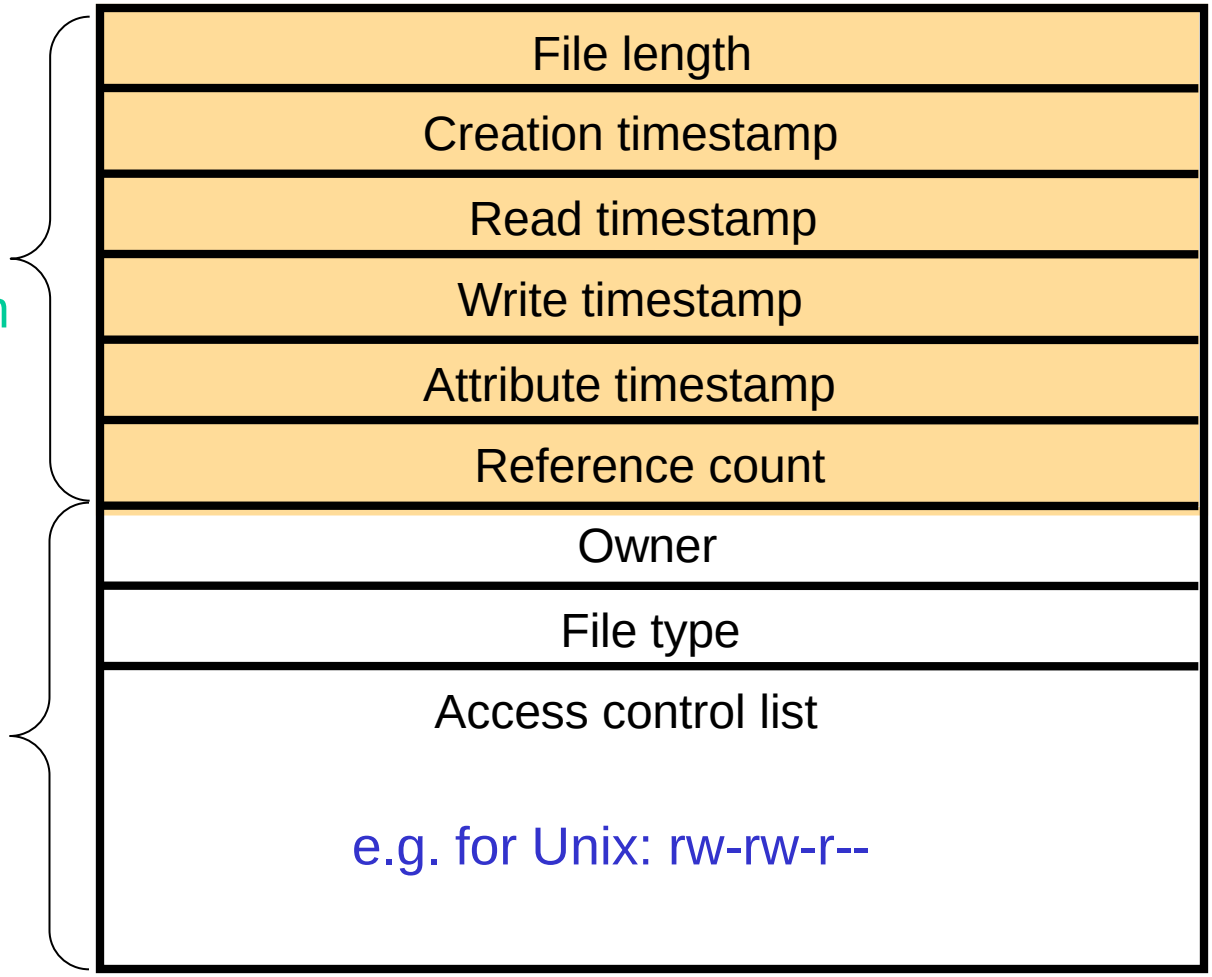
- File systems and databases: first candidates to be distributed and shared in a networked system
  - solutions for a long time for local area networks
- The management of a distributed file system is supported by two services:
  - name/directory service
  - file service
- It is essential to provide good performance
  - compromise between the local availability of information and consistency
    - *caching* (clients), distribution/replication (servers)

Introduction

- Properties of a file system:
  - it provides permanent information storage
  - it identifies the files in a namespace, normally structured (directories, subdirectories)
  - it manages the concurrent access to the information
  - in multi-user systems, it provides access protection (e.g., permissions to read/write/execute)
  - it manages the data and the attributes (metadata) of files

Introduction

File attributes:



Introduction

- Unix file system interface (system calls):

filedes = open(name, mode) *filedes = creat(name, mode)*	Opens an existing file with the given *name*. Creates a new file with the given *name*. Both operations deliver a file descriptor referencing the open file. The *mode* is *read*, *write* or both.
status = close(filedes)	Closes the open file *filedes*.
count = read(filedes, buffer, n) *count = write(filedes, buffer, n)*	Transfers *n* bytes from the file referenced by *filedes* to *buffer*. Transfers *n* bytes to the file referenced by *filedes* from buffer. Both operations deliver the number of bytes actually transferred and advance the read-write pointer.
pos = lseek(filedes, offset, whence)	Moves the read-write pointer to offset (relative or absolute, depending on *whence*).
status = unlink(name)	Removes the file *name* from the directory structure. If the file has no other names, it is deleted.
status = link(name1, name2)	Adds a new name (*name2*) for a file (*name1*).
status = stat(name, buffer)	Gets the file attributes for file *name* into *buffer*.

Introduction

- Unix file system:
  - operations offered by the Unix kernel
  - applications often use libraries:
    - C language standard library
    - Java classes for file management
  - the Unix file system stores information about the state of each open file:
    - it manages thus a table of open files
    - for each open file, it stores the current position (i.e., where the next read/write operation will be performed)
  - access control is done when opening the file:
    - by means of the *mode* parameter and the attributes of the file

Introduction

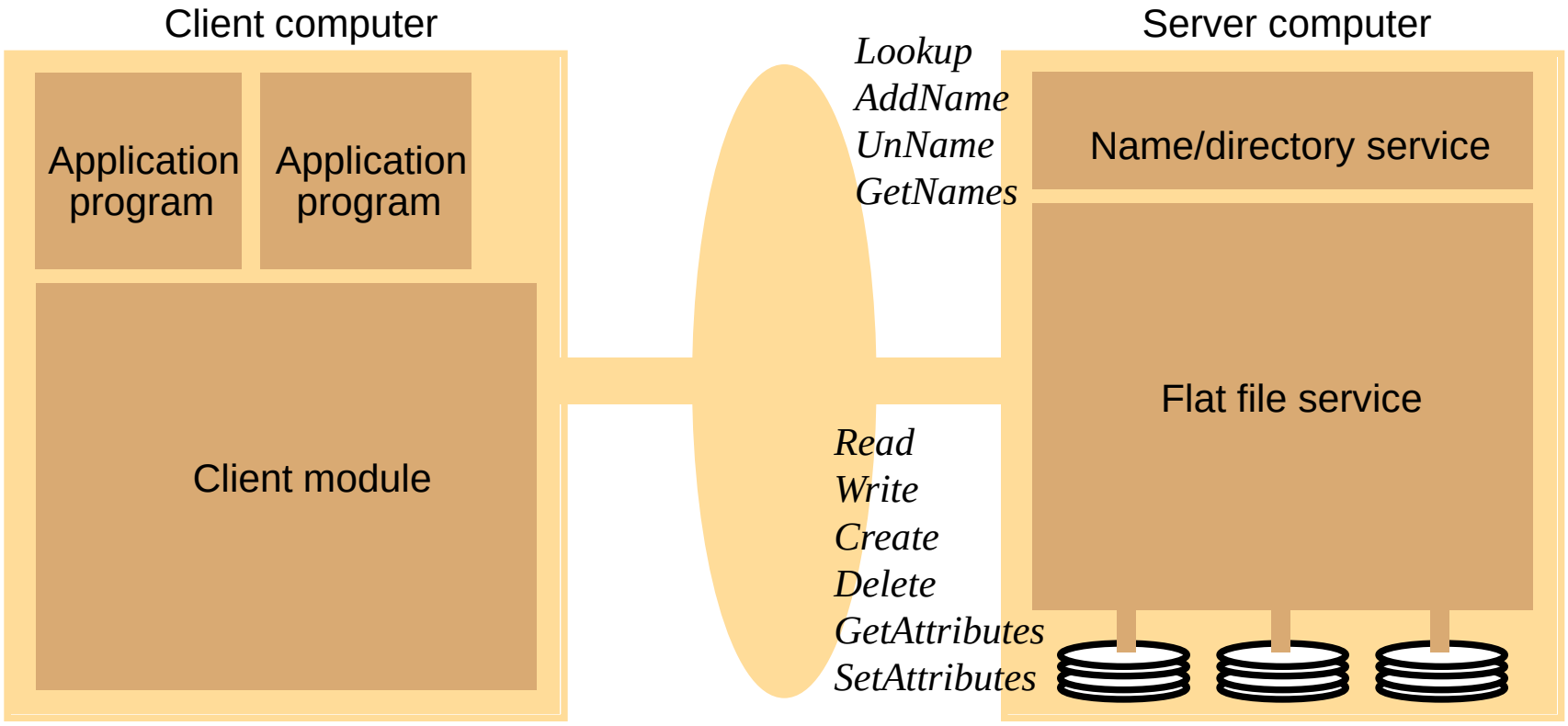
- (Desirable) Properties of distributed file systems:
  - transparency in file identification (unique namespace)
  - transparency in file location (to allow mobility)
  - scalability (*caching* to avoid bottlenecks)
  - robustness to failures (servers without state, idempotent operations, i.e., *f(f(x)) = f(x)*)
  - high availability and fault tolerance (replication, operation in disconnected mode)
  - consistency (Unix semantics)
  - security (remote authentication: credentials)

Introduction

- Characterization of the use of files:
  - most files are small (access unit)
  - writing is infrequent (*caching*, replication)
  - concurrent access is infrequent:
    - a reader and/or a writer most of the times, *n* readers and a writer sometimes, *n* readers and *m* writers very rarely
    - optimistic management of *caching* and replication
  - access is usually sequential, with a high degree of locality: anticipated read (*buffering*)
  - short life (e.g., temporary files: local management)
  - classes of files (e.g., executable files, rarely modified)

System model

- Client-server structure (3 modules):



Distributed file system architecture

System model

- Client module:
  - it is the local interface with the applications
  - it interprets system calls o files and generates requests (usually remote procedure calls, RPCs) for remote access
  - it knows the location of the name and file services
  - it manages local storage (*caching*)
- Flat file service:
  - it maintains the contents and attributes of files
  - *UFID*: unique file identifier
- Name/directory service:
  - it provides location transparency
  - it is a database with elements (*name*, *UFID*)

System model

- Identification and access to files:
  - using the path/name of the file, the client module requests the *UFID* of the file to the name service
    - information contained in the *UFID*: server (*S*), file inside the server (*F*), access rights over the file (*D*)
  - the subsequent operations on the file use its *UFID*
  - a mechanism for preventing the manipulation of the *UFID* is required
    - e.g., to claim access rights that a client does not have
  - the *UFID* is a credential (*capability*)

System model

- Authentication procedure (Amoeba):
  - when a file is created, the file service generates and stores a random number (*R*) associated with the file
  - the *UFID* issued by the name service to the client does not include *R*, but the result *C* of encoding *R* and the access rights for the client on the file *D*
  - in the operations on the file, the file service will encode the field *D* of the *UFID* presented by the client with the value *R* stored by the file service for that file, and if the result is equal to the field *C* of the *UFID* presented by the client, then the access is valid

Name/directory service

- Allows the client module to determine the location of a file (more specifically, the file *UFID*) from the symbolic name of the file (*path*)

Lookup(Dir, Name) -> FileId — throws *NotFound*	Locates the text name in the directory and returns the relevant UFID. If *Name* is not in the directory, throws an exception.
AddName(Dir, Name, FileId) — throws *NameDuplicate*	If *Name* is not in the directory, adds (*Name*, *FileId*) to the directory and updates the file's attribute record. If *Name* is already in the directory: throws an exception.
UnName(Dir, Name) — throws *NotFound*	If *Name* is in the directory: the entry containing *Name* is removed from the directory. If *Name* is not in the directory: throws an exception.
GetNames(Dir, Pattern) -> NameSeq	Returns all the text names in the directory that match the regular expression *Pattern*.

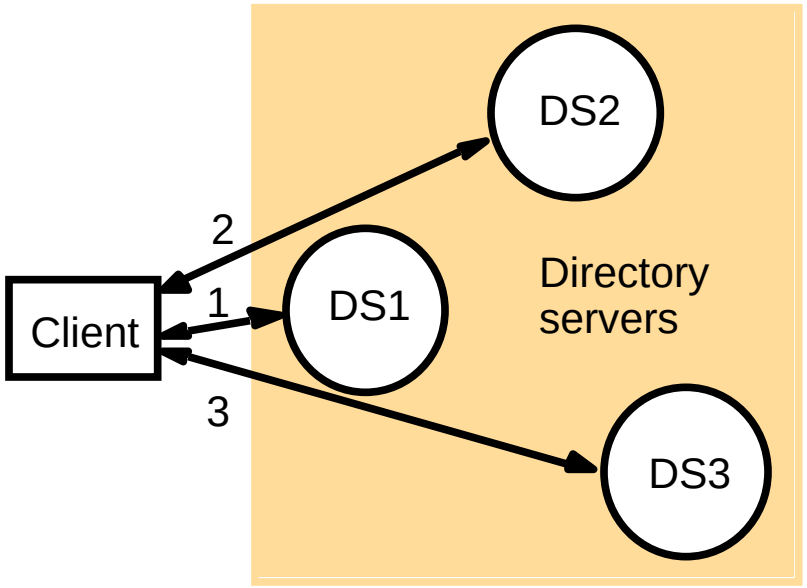
Name/directory service operations

Name/directory service

- From the point of view of name/path resolution, the directory structure is divided into domains
  - a domain can be associated with a part of the *path*
  - a name service can manage one or more domains
  - the client module maintains a table of (*domain*, *server*) entries
- Resolution of the *path* (also known as navigation):
  - iterative (controlled by the client or by the server)
  - recursive

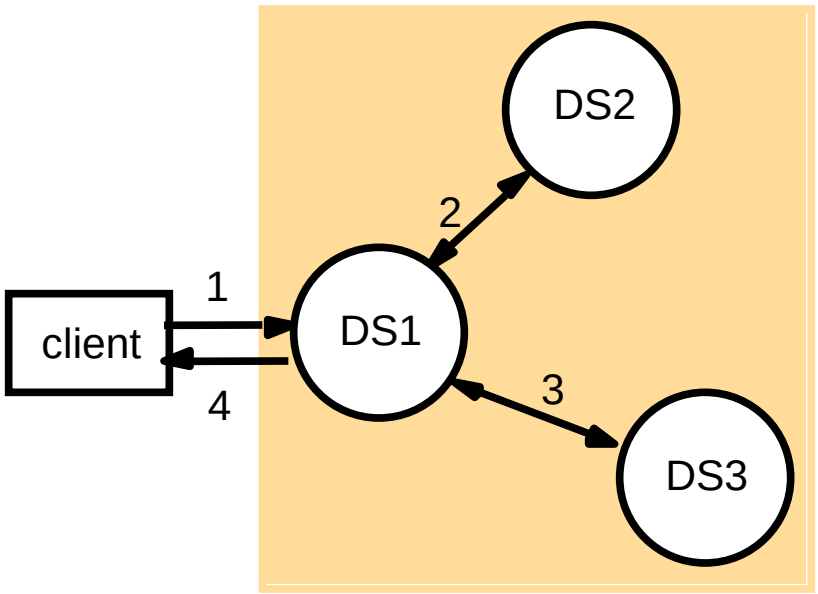
Name/directory service

- Iterative resolution of the *path* (controlled by the client):

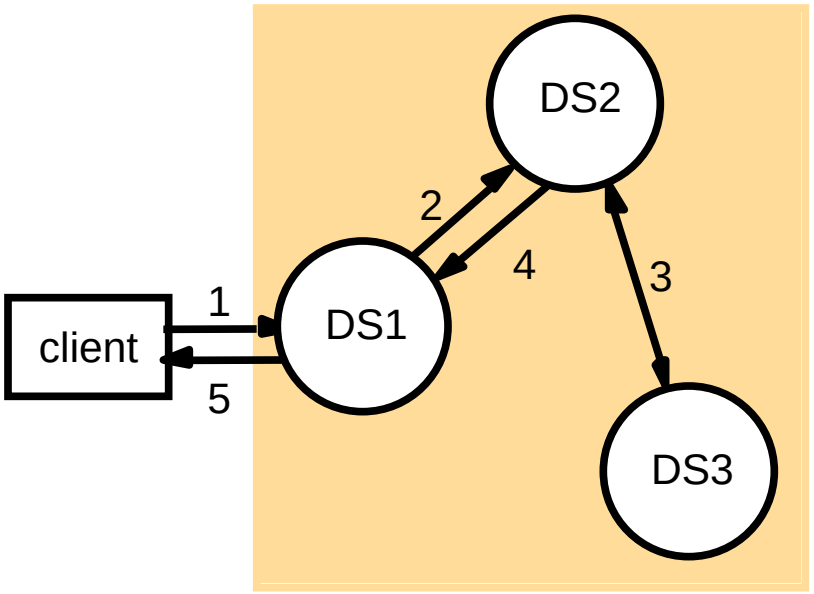


Name/directory service

- Iterative resolution of the *path* (controlled by the server) and recursive resolution:



Non-recursive
server-controlled



Recursive
server-controlled

Flat file service

- Objective: provide a semantics as close as possible to the one offered by centralized file systems, without penalizing performance (latency): *caching*

Read(FileId, i, n) -> Data — throws *BadPosition*	If $1 \leq i \leq Length(File)$: Reads a sequence of up to *n* items from a file starting at item *i* and returns it in *Data*.
Write(FileId, i, Data) — throws *BadPosition*	If $1 \leq i \leq Length(File)+1$: Writes a sequence of *Data* to a file, starting at item *i*, extending the file if necessary.
Create() -> FileId	Creates a new file of length 0 and delivers a UFID for it.
Delete(FileId)	Removes the file from the file store.
GetAttributes(FileId) -> Attr	Returns the file attributes for the file.
SetAttributes(FileId, Attr)	Sets the file attributes (only those attributes that are not shaded).

Flat file service operations

Flat file service

- Management of concurrent accesses: compromise between performance and consistency
  - Unix semantics (*one-copy*): centralized file systems
  - Session semantics (local copy on open, update on close)
  - Immutable files (atomic replacement)
  - Transactional semantics (*ACID* properties)
- Types of servers:
  - without state: *NFS* (*Network File System*)
    - the failure of a client does not affect the server
  - with state: *RFS* (*Remote File System*), *AFS*, *NFSv4*
    - the failure of a client affects the server

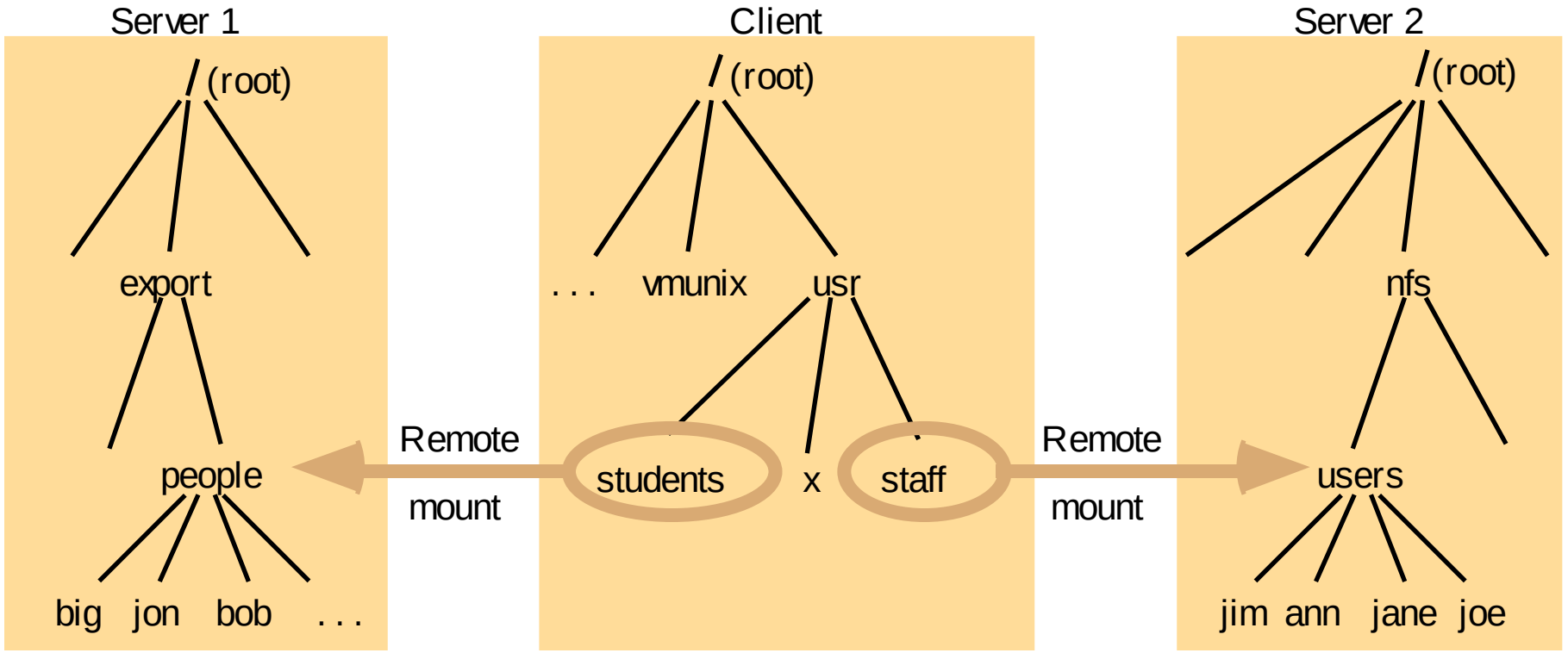
Flat file service

- *Caching*, *buffering*:
  - the client module stores local copies of (part of) the files, to reduce the number of remote communications
  - the server performs *buffer cache* of parts of the files
  - when serving requests, the server provides anticipated reading (*buffering*)
  - *Cache* consistency management:
    - *write-through*: when a block/file is modified, it is copied to the server. Delayed writing to improve performance. A validation mechanism is required to try to provide Unix semantics (controlled by the client or by the server)
    - *write-on-close*: session semantics
    - *centralized management*: Unix semantics (not scalable)

Network File System (NFS)

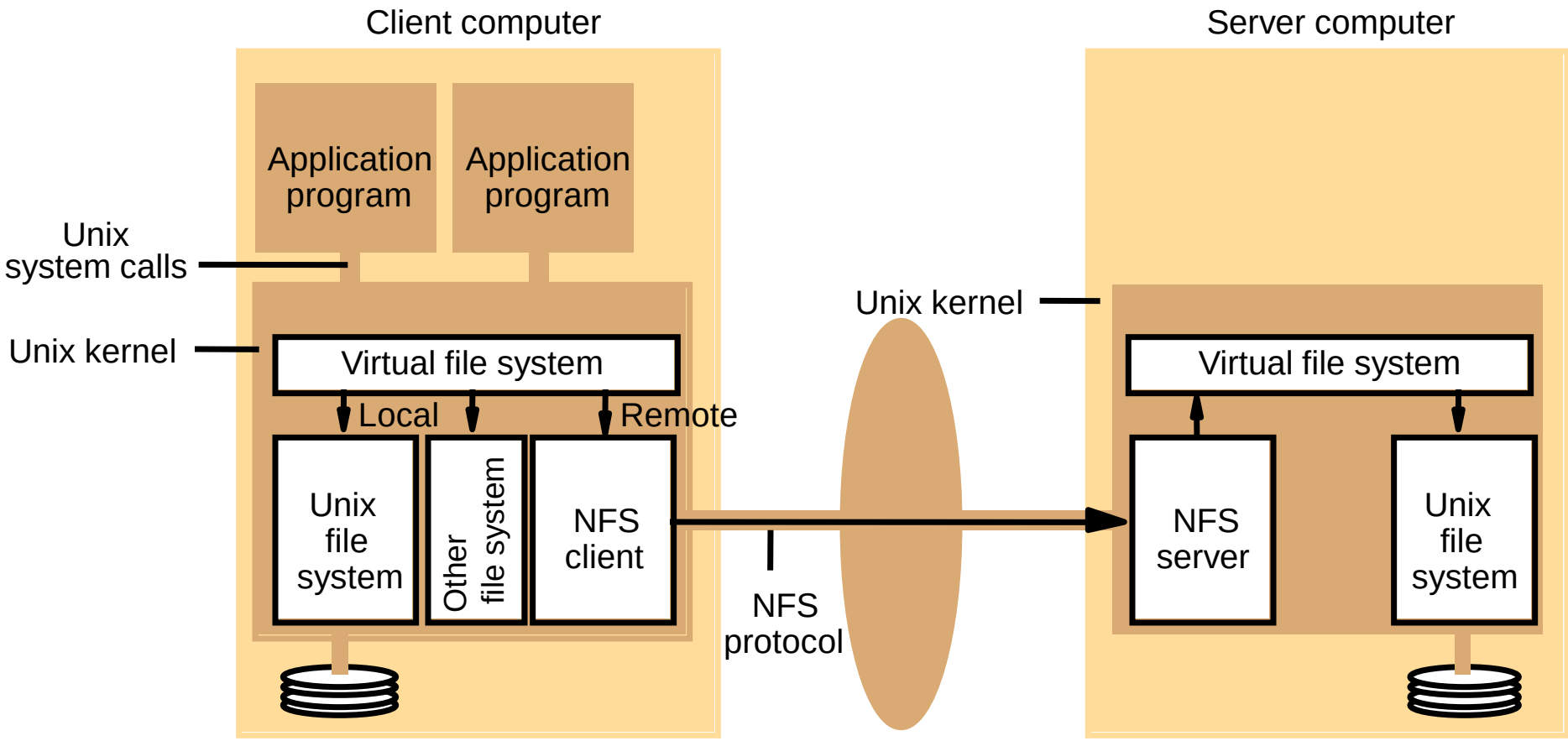
- Developed by *Sun Microsystems* (1980s)
  - initially only for Unix systems
  - currently: *VMS*, *Windows*, *MacOS*, *Linux*…
  - used in combination with *NIS* (*Network Information System*): information about the location of servers
- Servers export directories, that clients mount:
  - Unix: `/etc/exports` and `/etc/fstab` files
  - mounting can be done either in the boot of the operating system or on demand when a file is opened (*automounter* service)
- Applications use the Unix interface:
  - transparency in file identification/access

NFS – Example



Note: The file system mounted at */usr/students* in the client is actually the sub-tree located at */export/people* in Server 1; the file system mounted at */usr/staff* in the client is actually the sub-tree located at */nfs/users* in Server 2.

NFS – Architecture



NFS – Architecture

- The name/directory service and the flat file service are integrated in a single server (*NFS Server*)
- The file service is without state:
  - failures are managed much simpler
  - access control: access rights are checked in each request
- Operations are idempotent (not all, e.g., *create*)
- The client module performs block/file *caching*:
  - typically blocks of 16 Kbytes (anticipated reading)
  - *write-through* semantics, with delayed writings. The client module periodically checks the validity of the cached blocks

NFS – Architecture

- Management of concurrent accesses:
  - Unix semantics (limited, due to *caching*)
- Protocols:
  - MOUNT: mounting/unmounting directories
  - NFS: access to remote files
  - both are based on the RPC protocol
    - RPC works with both TCP and UDP
    - consistent representation of the information: XDR format
- Implementation: *VFS* (*Virtual File System*)
  - table of *v-nodes* of open files: if the file is local, the *v-node* points to an *inode*, and if it is remote to an *r-node* (*remote inode*), which is managed by the client module

NFS – Operations

lookup(dirfh, name) -> fh, attr	Returns file handle and attributes for the file *name* in the directory *dirfh*.
create(dirfh, name, attr) -> newfh, attr	Creates a new file name in directory *dirfh* with attributes *attr* and returns the new file handle and attributes.
remove(dirfh, name) status	Removes file name from directory *dirfh*.
getattr(fh) -> attr	Returns file attributes of file *fh*. (Similar to the Unix *stat* system call.)
setattr(fh, attr) -> attr	Sets the attributes (mode, user id, group id, size, access time and modify time of a file). Setting the size to 0 truncates the file.
read(fh, offset, count) -> attr, data	Returns up to *count* bytes of data from a file starting at *offset*. Also returns the latest attributes of the file.
write(fh, offset, count, data) -> attr	Writes *count* bytes of data to a file starting at *offset*. Returns the attributes of the file after the write has taken place.
rename(dirfh, name, todirfh, toname) -> status	Changes the name of file *name* in directory *dirfh* to *toname* in directory to *todirfh*
link(newdirfh, newname, dirfh, name) -> status	Creates an entry *newname* in the directory *newdirfh* which refers to file *name* in the directory *dirfh*.

NFS – Operations

symlink(newdirfh, newname, string) -> status	Creates an entry *newname* in the directory *newdirfh* of type symbolic link with the value *string*. The server does not interpret the *string* but makes a symbolic link file to hold it.
readlink(fh) -> string	Returns the string that is associated with the symbolic link file identified by *fh*.
mkdir(dirfh, name, attr) -> newfh, attr	Creates a new directory *name* with attributes *attr* and returns the new file handle and attributes.
rmdir(dirfh, name) -> status	Removes the empty directory *name* from the parent directory *dirfh*. Fails if the directory is not empty.
readdir(dirfh, cookie, count) -> entries	Returns up to *count* bytes of directory entries from the directory *dirfh*. Each entry contains a file name, a file handle, and an opaque pointer to the next directory entry, called a *cookie*. The *cookie* is used in subsequent *readdir* calls to start reading from the following entry. If the value of *cookie* is 0, reads from the first entry in the directory.
statfs(fh) -> fsstats	Returns file system information (such as block size, number of free blocks and so on) for the file system containing a file *fh*.

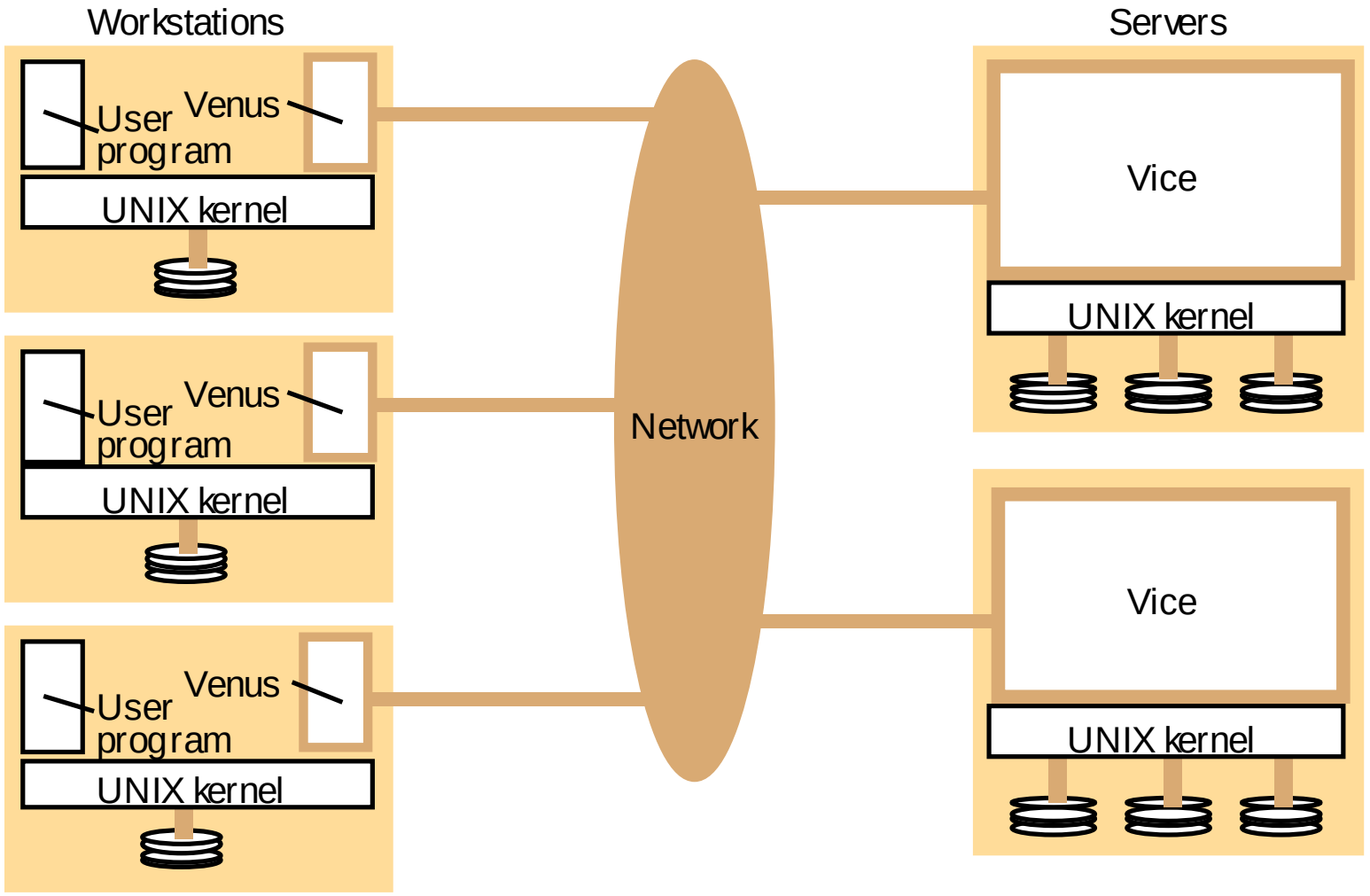
NFS – Limitations

- Consistency: providing Unix semantics is difficult
- Transparency in location is not automatic
- A mechanism is needed to provide exclusive access to files
  - in Unix the *lockd* service is used
- Servers cannot be replicated
  - only for reading accesses
  - updates must be managed manually
- Limited scalability
  - designed for local area networks

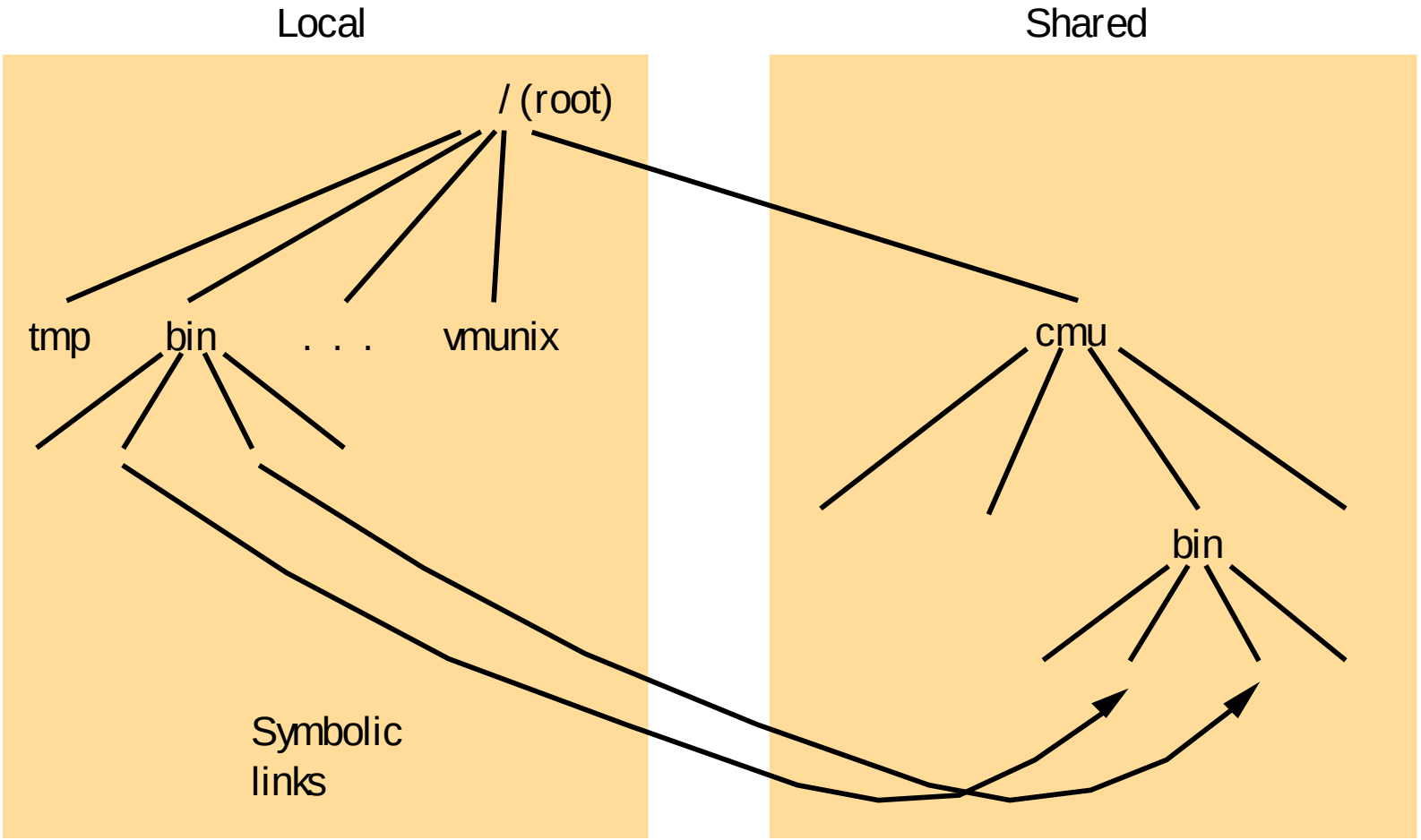
Andrew File System (AFS)

- Carnegie Mellon University, 1980s
- Supports information sharing on a large scale
  - 10000 workstations and servers running Unix
- Access transparency: client programs use normal Unix file primitives
- *Scalability* as the most important design goal:
  - *whole-file serving* by AFS servers to client nodes
  - *whole-file caching* (on local disk) in client nodes
- Uses a session semantics (*write-on-close*)
  - assumption: concurrent updates are very rare

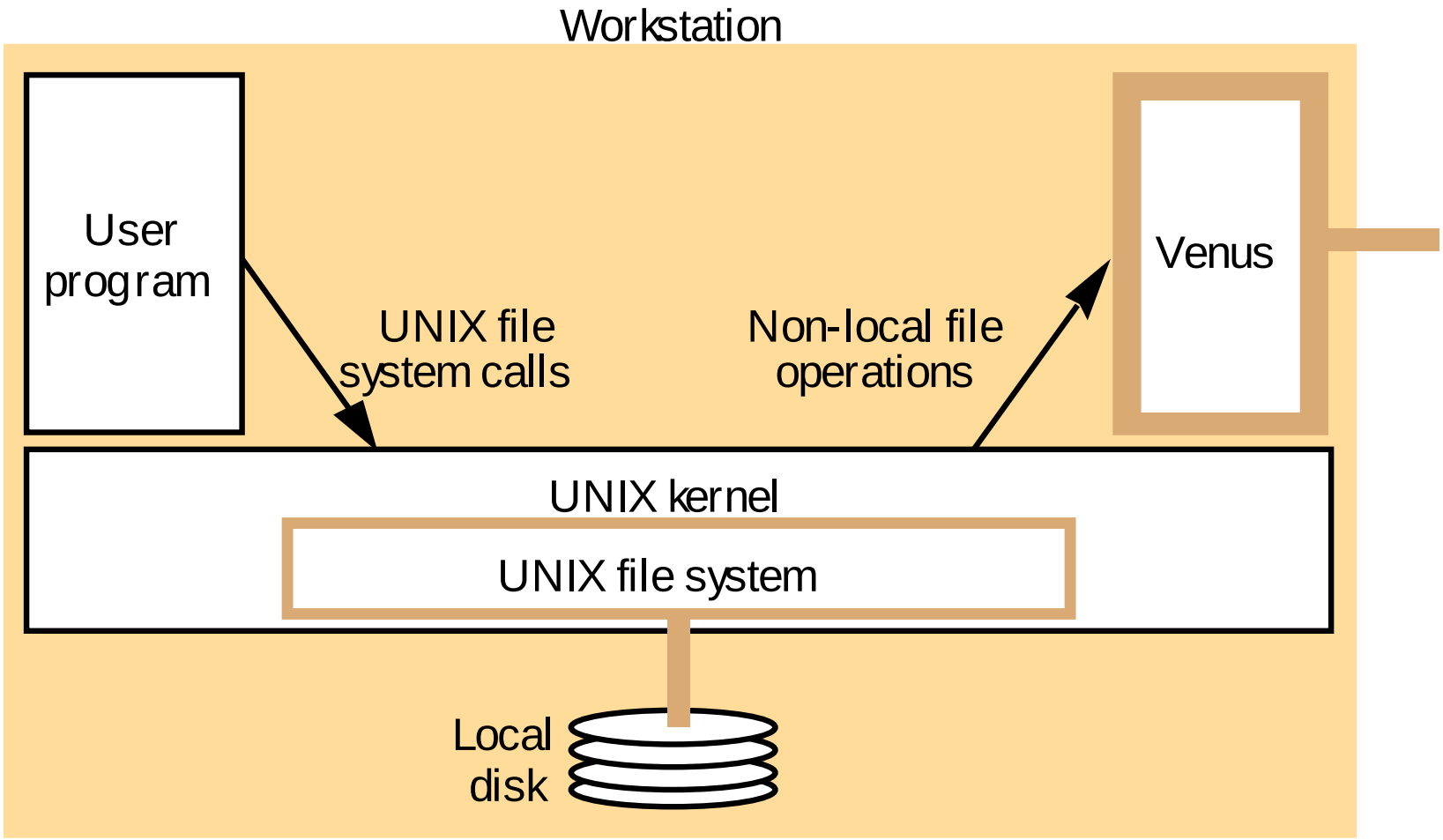
AFS – Architecture



AFS – Architecture



AFS – Architecture



AFS – Cache consistency

- When Vice supplies a copy of a file to a Venus process it also provides a *callback promise*
- When a server performs a request to update a file it notifies all of the Venus processes to which it has issued callback promises by sending a *callback* to each
  - servers are not stateless!
  - the callback-based mechanism offers the most scalable approach
- On *open*: if Venus has a *valid* copy it can open it directly, otherwise it has to fetch a fresh copy from the Vice server
- When Venus is restarted after a failure or a shut-down, the validity of the files in the cache must be checked, since some callbacks may have been missed

AFS – Operations

User process	*UNIX kernel*	*Venus*	*Net*	*Vice*
open(FileName, mode)	If *FileName* refers to a file in shared file space, pass the request to Venus.	Check list of files in local cache. If not present or there is no valid *callback promise*, send a request for the file to the Vice server that is custodian of the volume containing the file.	→	Transfer a copy of the file and a *callback promise* to the workstation. Log the callback promise.
	Open the local file and return the file descriptor to the application.	Place the copy of the file in the local file system, enter its local name in the local cache list and return the local name to UNIX.	←	
read(FileDescriptor, Buffer, length)	Perform a normal UNIX read operation on the local copy.			
write(FileDescriptor, Buffer, length)	Perform a normal UNIX write operation on the local copy.			
close(FileDescriptor)	Close the local copy and notify Venus that the file has been closed.	If the local copy has been changed, send a copy to the Vice server that is the custodian of the file.	→	Replace the file contents and send a *callback* to all other clients holding *callback promises* on the file.

Coda

- Descendant of AFS that is substantially more resilient to server and network failures
- Coda (*COnstant Data Availability*): AFS + ...
  - replication (for load-balancing and fault-tolerance)
  - support for mobile users: *disconnected operation* mode
- Directories are replicated in several Vice servers
- When Venus is disconnected, it uses local versions of files. When Venus reconnects, it reintegrates using an optimistic update scheme

Common Internet File System

- Service Message Block (SMB) was originally developed by IBM for DOS in the 1980s, and then modified by Microsoft in the 1990s for Windows
  - LAN Manager Server (server side)
  - LAN Manager Workstation (client side)
- Common Internet File System (CIFS) is based on Microsoft's SMB for Internet/intranet file sharing
- *Samba* is an open source reimplementation of SMB/CIFS for Unix-like systems (e.g., Linux)

Service Message Block (SMB)

- SMB uses client *locks* to solve cache consistency
  - the server is state-full!
- A client can open a file and lock it; all read and write operations are done in the client cache
- A read-only lock will allow multiple clients to cache and read a file
- Locks can be revoked by the server forcing the client to flush any changes